

Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Monterrey



TE3001B

Fundamentación de robótica

Mini-reto: Crear un controlador para regular velocidad de motor usando ROS

Alumnos:

Josue Ureña Valencia	IRS A01738940
César Arellano Arellano	IRS A00839373
Jose Eduardo Sanchez Martinez	IRS A01738476
Rafael André Gamiz Salazar	IRS A00838280

Fecha de entrega:

04 de marzo del 2026

Índice

Introducción.....	3
Fase 1. Controlador PID y modificación de parámetros.....	4
Arquitectura del sistema.....	4
Nodo motor.....	4
Librerías utilizadas.....	5
Definición de variable globales ROS.....	5
Macros para control de errores y temporización.....	6
Máquina de estados del agente.....	6
Configuración de Pines, PWM y de funciones.....	7
Callback del suscriptor.....	7
Setup del sistema.....	8
Función Loop y gestión del nodo.....	9
Función create_entities().....	10
Nodo pwm_pub.....	13
Importar librerías.....	13
Definición del nodo, publicador y timer.....	13
Funcion publish_pwm(self).....	14
Función main().....	14

Video:

Introducción

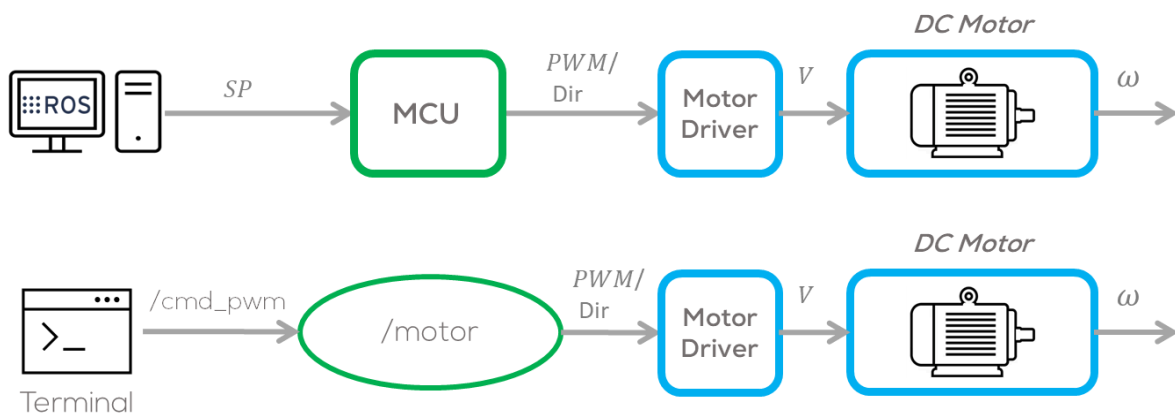
En este documento se describe el paso a paso de la implementación técnica del Mini Challenge asignado a la semana 3 del curso. El objetivo de este mini challenge es controlar la velocidad de un motor DC utilizando ROS2 como sistema de comunicación distribuido y un microcontrolador ESP32 ejecutando micro-ROS. El sistema permite modificar la velocidad y dirección del motor mediante la publicación de un valor numérico en el tópico `/cmd_pwm`.

Fase 1. Controlador PID y modificación de parámetros

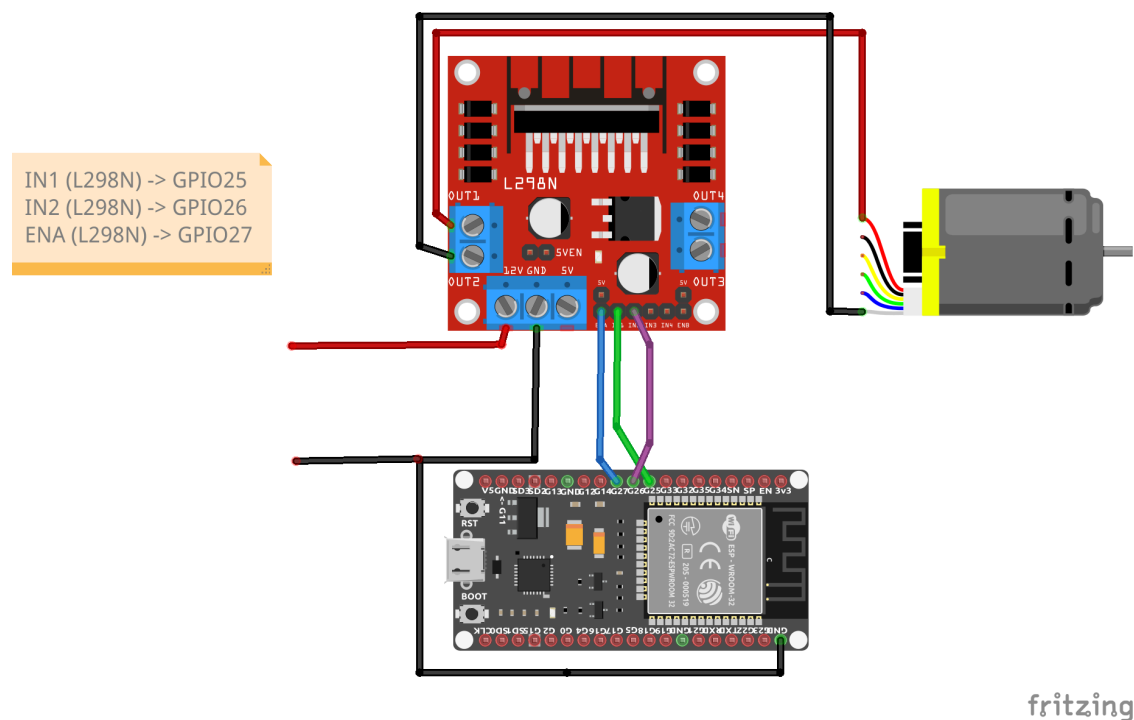
Arquitectura del sistema

El sistema está compuesto por los siguientes elementos:

- Computadora con ROS2
- Nodo publicador en Python
- ESP32 ejecutando micro-ROS
- Driver L298N
- Motor DC 12V



El nodo en la computadora publica comandos de velocidad (entre -1.0 y 1.0), mientras que el ESP32 recibe estos comandos y genera las señales eléctricas necesarias para controlar el motor. El diagrama de conexión utilizado es el siguiente:



Nodo motor

El ESP32 ejecuta un nodo suscriptor que:

- Se conecta al agente micro-ROS
- Se suscribe al tópico `/cmd_pwm`
- Recibe mensajes tipo `Float32`
- Convierte el valor recibido en:
 - Señal de dirección
 - Señal PWM

Para la implementación, se utilizó como base el código que se mostró en clase, el cual es “micro_ros_subscriber_reconnection.ino”, y solo lo adaptamos a las necesidades del reto. A continuación se detalla el código utilizado:

Librerías utilizadas

```
C/C++
// Include Libraries to be used
#include <micro_ros_arduino.h> //micro-ros-arduino library
#include <rcl/rcl.h> //Core ROS 2 Client Library (RCL) for node
management.
#include <rcl/error_handling.h> //Error handling utilities for Micro-ROS.
#include <rcl/rcl.h> //Micro-ROS Client library for embedded
devices.
#include <rcl/executor.h> //Micro-ROS Executor to manage callbacks
#include <std_msgs/msg/float32.h> //Predefined ROS 2 message type
#include <rmw_microros/rmw_microros.h> //ROS Middleware for Micro-ROS,
provides functions for interfacing Micro-ROS with DDS.
#include <stdio.h> //Standard I/O library for debugging.
```

Estas librerías permiten ejecutar un nodo ROS en el ESP32 usando micro-ROS, además de crear el suscriptor, usar mensajes tipo Float32, y gestionar memoria y ejecución.

Definición de variable globales ROS

```
C/C++
//Declare nodes to be used
rcl_node_t node; //Represents a ROS 2 Node running on the
microcontroller.
```

```

//Instantiate executor and its support classes
rclc_executor_t executor;    //Manages task execution (timers, callbacks,
etc.).
rclc_support_t support;     //Data structure that holds the execution
context of Micro-ROS, including its communication state, memory management,
and initialization data.
rcl_allocator_t allocator;  //Manages memory allocation.

//Declare Subscribers to be used
rcl_subscription_t subscriber;

//Declare Messages to be used
std_msgs__msg_Float32 msg;  //Defines a message of type float32.

```

Estas estructuras nos permiten, crear el nodo (**node**), gestionar la ejecución de callback (**excecutor**), administrar la memoria (**allocator**), crear el suscriptor al tópico (**subscriber**) y almacenar el mensaje recibido (**msg**).

Macros para control de errores y temporización

```

C/C++
//Define Macros to be used
//Executes fn and returns false if it fails.
#define RCCHECK(fn) { rcl_ret_t temp_rc = fn; if((temp_rc !=
RCL_RET_OK)){return false;}}
// Executes fn, but ignores failures.
#define RCSOFTCHECK(fn) { rcl_ret_t temp_rc = fn; if((temp_rc !=
RCL_RET_OK)){}}
// Executes a given statement (X) periodically every MS milliseconds
#define EXECUTE_EVERY_N_MS(MS, X) do { \
    static volatile int64_t init = -1; \
    if (init == -1) { init = uxr_millis();} \
    if (uxr_millis() - init > MS) { X; init = uxr_millis();} \
} while (0)\

```

Estas macros nos permiten manejar errores y controlar tiempos de ejecución en el sistema, RCCHECK detiene la ejecución si ocurre un error crítico, RCSOFTCHECK ignora errores no graves, y EXECUTE_EVERY_N_MS permite ejecutar instrucciones periódicamente sin bloquear el programa. En conjunto, estas macros hacen que el nodo en micro-ROS sea más estable y robusto.

Máquina de estados del agente

```
C/C++  
//Defines State Machine States  
enum states {  
    WAITING_AGENT,  
    AGENT_AVAILABLE,  
    AGENT_CONNECTED,  
    AGENT_DISCONNECTED  
} state;
```

Dentro del ESP32 se implementa una máquina de estados para manejar la conexión con el agente micro-ROS. Los estados significan lo siguiente:

- WAITING_AGENT: espera a que el agente esté disponible
- AGENT_AVAILABLE: intenta crear entidades ROS
- AGENT_CONNECTED: ejecuta el nodo y procesa los mensajes
- AGENT_DISCONNECTED: destruye entidades si se pierde la conexión

Esto garantiza que el sistema sea robusto ante desconexiones.

Configuración de Pines, PWM y de funciones

```
C/C++  
//Specifies GPIO pin 27 for controlling an PWM  
#define PWM_PIN 27 //DEFINE PWM_PIN  
#define PWM_FRQ 980 //Define PWM Frequency  
#define PWM_RES 8 //Define PWM Resolution  
#define PWM_CHNL 0 //Define Channel  
#define MSG_MIN_VAL -1 //Define min input value  
#define MSG_MAX_VAL 1 //Define max input value  
#define IN1 25  
#define IN2 26  
  
//Create entity functions  
bool create_entities();  
void destroy_entities();
```

Aquí se definen el pin PWM para el control de la velocidad, el pin IN1 e IN2 para controlar la dirección, la frecuencia PWM de 980 Hz, y una resolución de 8 bits, lo que nos permite que el duty cycle varíe entre 0 y 255. Además, se declaran las funciones `create_entities()` y `destroy_entities()`, que se usarán más adelante.

Callback del suscriptor

```
C/C++
void subscription_callback(const void * msgin)
{
    const std_msgs__msg__Float32 * msg = (const std_msgs__msg__Float32
*)msgin;
    float cmd = constrain(msg->data, MSG_MIN_VAL, MSG_MAX_VAL);
    // Obtener magnitud
    float magnitude = abs(cmd);
    // Convertir a duty cycle
    uint32_t duty = (uint32_t)((pow(2, PWM_RES) - 1) * magnitude);
    // Controlar dirección
    if(cmd > 0){
        digitalWrite(IN1, HIGH);
        digitalWrite(IN2, LOW);
    } else if(cmd < 0){
        digitalWrite(IN1, LOW);
        digitalWrite(IN2, HIGH);
    } else{
        digitalWrite(IN1, HIGH);
        digitalWrite(IN2, HIGH);
        duty = 0;
    }
    ledcWrite(PWM_CHNL, duty);
}
```

Esta función callback se ejecuta cada vez que se recibe un mensaje en el tópico `cmd_pwm`. Luego, el valor recibido se satura en el intervalo $[-1,1]$, evitando errores por valores fuera de rango. Posteriormente se obtiene la magnitud del valor, para la velocidad y se convierte a un duty cycle entre 0 y 255 para con ello establecer el PWM. Después, dependiendo del signo del valor recibido, se determina la dirección de giro:

- `cmd > 0`: girar hacia delante
- `cmd < 0`: giro hacia atrás
- `cmd == 0`: el motor está en modo freno

Una vez definido el giro, y el valor PWM, el ESP32 genera la señal PWM con el duty cycle calculador y la envía hacia el puente H a través del pin correspondiente.

Setup del sistema

```
C/C++
void setup() {
    set_microros_transports(); // Initializes communication between ESP32 and
    the ROS 2 agent (Serial).
    //Setup Microcontroller Pins
    pinMode(PWM_PIN, OUTPUT);
    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
    ledcSetup(PWM_CHNL, PWM_FRQ, PWM_RES); //Setup the PWM
    ledcAttachPin(PWM_PIN, PWM_CHNL);      //Setup Attach the Pin to the
    Channel
}
```

En el setup se realiza la comunicación con micro-ROS, se configuran los pines como salida, así como la configuración del módulo PWM.

Función Loop y gestión del nodo

```
C/C++
void loop() {
    switch (state) {

        case WAITING_AGENT:
            EXECUTE_EVERY_N_MS(500, state = (RMW_RET_OK ==
            rmw_uros_ping_agent(100, 1)) ? AGENT_AVAILABLE : WAITING_AGENT);
            break;

        case AGENT_AVAILABLE:
            state = (true == create_entities()) ? AGENT_CONNECTED : WAITING_AGENT;
            if (state == WAITING_AGENT) {
                destroy_entities();
            };
            break;

        case AGENT_CONNECTED:
            EXECUTE_EVERY_N_MS(200, state = (RMW_RET_OK ==
            rmw_uros_ping_agent(100, 1)) ? AGENT_CONNECTED : AGENT_DISCONNECTED);
            if (state == AGENT_CONNECTED) {
                rcl_executor_spin_some(&executor, RCL_MS_TO_NS(100));
            }
    }
}
```

```

    }
    break;

case AGENT_DISCONNECTED:
    destroy_entities();
    state = WAITING_AGENT;
    break;

default:
    break;
}
}

```

En el `loop()` se ejecuta la máquina de estados, dentro, la función `rclcpp_executor_spin_some()` es quien permite revisar constantemente si llegaron nuevos mensajes así como ejecutar la función de callback cuando sea correspondiente.

Función `create_entities()`

```

C/C++
bool create_entities()
{
    //Initializes memory allocation for Micro-ROS operations.
    allocator = rcl_get_default_allocator();

    //Creates a ROS 2 support structure to manage the execution context.
    RCCHECK(rclcpp_support_init(&support, 0, NULL, &allocator));

    // create node
    RCCHECK(rclcpp_node_init_default(&node, "motor", "", &support));

    // create subscriber
    RCCHECK(rclcpp_subscription_init_default(
        &subscriber,
        &node,
        ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, Float32),
        "cmd_pwm"));

    // create zero initialised executor (no configured) to avoid memory
    problems
    executor = rclcpp_executor_get_zero_initialized_executor();
    // Initializes the Micro-ROS Executor, which manages tasks and callbacks.
    RCCHECK(rclcpp_executor_init(&executor, &support.context, 1, &allocator));
    // Register suscription with executor
    RCCHECK(rclcpp_executor_add_subscription(&executor, &subscriber, &msg,
        &subscription_callback, ON_NEW_DATA));
}

```

```
    return true;
}
```

En esta función se crean todas las entidades de ROS que necesita el ESP32, se crea el allocator, en el contexto de ROS, se crea el nodo `motor`, el suscriptor `cmd_pwm`, que recibe mensajes de Float32 y además se crea y se inicializa el executor.

Función `destroy_entities()`

```
C/C++
void destroy_entities()
{
    rmw_context_t * rmw_context =
    rcl_context_get_rmw_context(&support.context);
    (void) rmw_uos_set_context_entity_destroy_session_timeout(rmw_context,
    0);

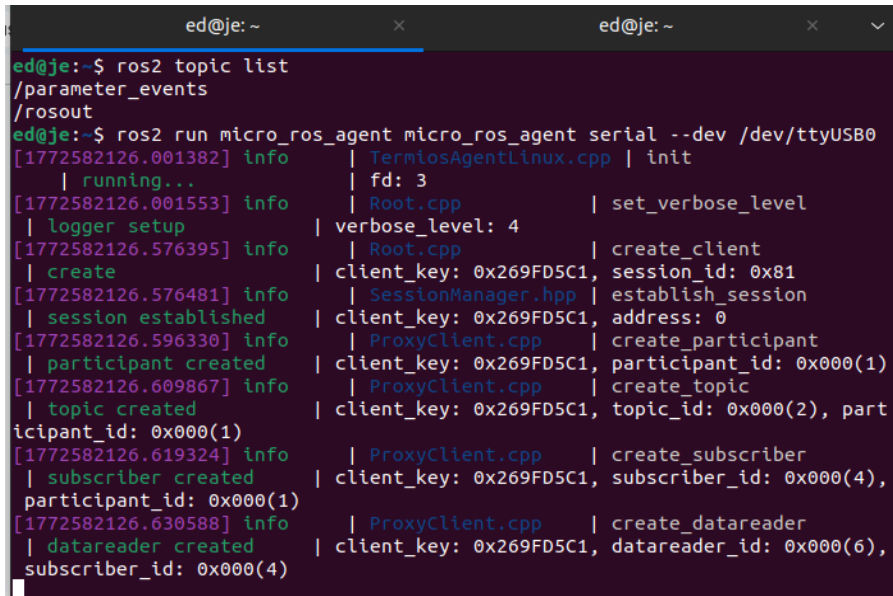
    rcl_subscription_fini(&subscriber, &node);
    rcl_executor_fini(&executor);
    rcl_node_fini(&node);
    rcl_support_fini(&support);
}
```

Esta función permite destruir todas las entidades de ROS que se generaron anteriormente, esto con el motivo de liberar memoria, cerrar la sesión DDS, evitar fugas de memoria y permitir una reconexión limpia.

Una vez terminado el código, se carga al ESP32 y se inicializa el agente en nuestra computadora para que permita la conexión entre la terminal y el nodo del ESP32. El agente se inicializa con el siguiente comando:

```
Shell
ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyUSB0
```

Y se puede observar cómo se detecta la conexión con el nodo **motor**.

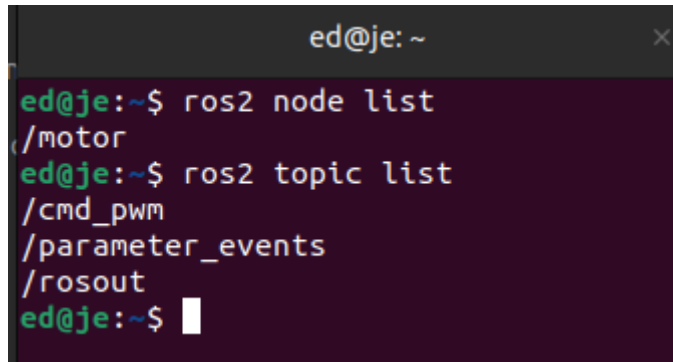


```
ed@je: ~  
ed@je:~$ ros2 topic list  
/parameter_events  
/rosout  
ed@je:~$ ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyUSB0  
[1772582126.001382] info | TermiosAgentLinux.cpp | init  
| running...  
[1772582126.001553] info | fd: 3  
[1772582126.001553] info | Root.cpp | set_verbose_level  
| logger setup | verbose_level: 4  
[1772582126.576395] info | Root.cpp | create_client  
| create | client_key: 0x269FD5C1, session_id: 0x81  
[1772582126.576481] info | SessionManager.hpp | establish_session  
| session established | client_key: 0x269FD5C1, address: 0  
[1772582126.596330] info | ProxyClient.cpp | create_participant  
| participant created | client_key: 0x269FD5C1, participant_id: 0x000(1)  
[1772582126.609867] info | ProxyClient.cpp | create_topic  
| topic created | client_key: 0x269FD5C1, topic_id: 0x000(2), participant_id: 0x000(1)  
[1772582126.619324] info | ProxyClient.cpp | create_subscriber  
| subscriber created | client_key: 0x269FD5C1, subscriber_id: 0x000(4), participant_id: 0x000(1)  
[1772582126.630588] info | ProxyClient.cpp | create_datareader  
| datareader created | client_key: 0x269FD5C1, datareader_id: 0x000(6), subscriber_id: 0x000(4)
```

Una vez hecha la conexión, los siguientes comandos nos sirven para verificar que el nodo **motor** y el tópico “**cmw_pwm**”, se encuentran activos.

Shell

```
ros2 node list  
ros2 topic list
```



```
ed@je: ~  
ed@je:~$ ros2 node list  
/motor  
ed@je:~$ ros2 topic list  
/cmd_pwm  
/parameter_events  
/rosout  
ed@je:~$
```

Y mediante el siguiente comando se puede publicar un valor en el tópico **cmd_pwm** desde la terminal de la computadora, el ESP32 lo detecta y transforma en un valor de PWM para el motor:

Shell

```
ros2 topic pub /cmd_pwm std_msgs/msg/Float32 "data: 1.0"
```

Ejemplos del movimiento del motor.

Motor gira hacia adelante:

```
ed@je: ~  
ed@je:~$ ros2 topic pub /cmd_pwm std_msgs/msg/Float32 "data: 0.9"  
publisher: beginning loop  
publishing #1: std_msgs.msg.Float32(data=0.9)  
publishing #2: std_msgs.msg.Float32(data=0.9)
```

Motor se detiene:

```
ed@je: ~  
^Ced@je:~$ ros2 topic pub /cmd_pwm std_msgs/msg/Float32 "data: 0.0"  
publisher: beginning loop  
publishing #1: std_msgs.msg.Float32(data=0.0)  
publishing #2: std_msgs.msg.Float32(data=0.0)
```

Motor gira hacia atrás:

```
ed@je: ~  
^Ced@je:~$ ros2 topic pub /cmd_pwm std_msgs/msg/Float32 "data: -1.0"  
publisher: beginning loop  
publishing #1: std_msgs.msg.Float32(data=-1.0)  
publishing #2: std_msgs.msg.Float32(data=-1.0)
```

Video de la ejecución de las 3 ejecuciones: [video](#)

Después de esta demostración, se optó por crear un nodo de ROS2 para que publique constantemente valores entre -1 y 1, para así nosotros evitar enviar el valor por la terminal.

Nodo pwm_pub

Este nodo publica un valor PWM aleatorio entre -1 y 1 cada 5 segundos en el tópico `cmd_pwm`, se escogió el valor de 5 segundos con el motivo de poder observar correctamente la ejecución del PWM correspondiente.

Importar librerías

```
Python  
import rclpy  
from rclpy.node import Node  
from std_msgs.msg import Float32  
import random
```

Se importan las librerías necesarias, entre ellas la que nos permite usar nodos, Float32 y valores random.

Definición del nodo, publicador y timer

```
Python
class PwmPublisher(Node):

    def __init__(self):
        super().__init__('pwm_pub')
        self.publisher_ = self.create_publisher(Float32, '/cmd_pwm', 10)
        self.timer = self.create_timer(5.0, self.publish_pwm)

        self.get_logger().info("PWM Random Publisher iniciado")
```

Aquí se crea el nodo `pwm_pub`, el publicador al tópico `cmd_pwm` que envía mensajes Float32, y se crea el timer, que hace que cada 5 segundos se ejecute la función `self.publish_pwm`.

Funcion `publish_pwm(self)`

```
Python
def publish_pwm(self):
    msg = Float32()
    self.value = random.uniform(-1.0, 1.0)
    msg.data = float(self.value)
    self.publisher_.publish(msg)
    self.get_logger().info(f'Publicando: {msg.data:.3f}')
```

Esta función se ejecuta cada 5 segundos, crea el mensaje vacío, genera un número decimal entre -1.0 y 1.0, lo guarda en el mensaje y lo publica en el tópico `cmd_pwm`.

Función `main()`

```
Python
def main(args=None):
    rclpy.init(args=args)
    node = PwmPublisher()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
```

```
main()
```

El main permite inicializar ROS2, arranca el nodo, lo mantiene en ejecución, y cuando termina, lo cierra correctamente.

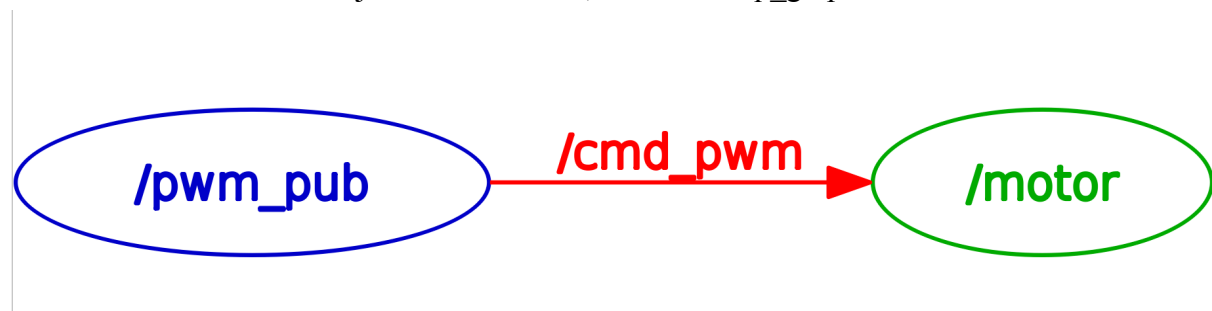
Para probar el nodo creado, se ejecuta con el siguiente comando.

```
Shell  
ros2 run pwm_publisher pwm_pub
```

Y se puede observar como el nodo publica los valores correspondientes.

```
ed@je: ~  
ed@je:~/ros2_challenge_mcr2/ros2_challenge1$ ros2 run pwm_publisher pwm_pub  
[INFO] [1772586523.241592177] [pwm_pub]: PWM Random Publisher iniciado  
[INFO] [1772586528.236409888] [pwm_pub]: Publicando: 0.561  
[INFO] [1772586533.236422453] [pwm_pub]: Publicando: 0.711  
[INFO] [1772586538.236547246] [pwm_pub]: Publicando: -0.061  
[INFO] [1772586543.236474749] [pwm_pub]: Publicando: -0.638  
[INFO] [1772586548.236500000] [pwm_pub]: Publicando: 0.604
```

Otra forma de observar la ejecución del nodo, es usando rqt_grap:



Se puede observar como el nodo `/pwm_pub`, ubicado en nuestra computadora, publica a través del canal `/cmd_pwm` hacia el nodo `/motor`, que se encuentra en el ESP32.

Este comportamiento se puede observar en el siguiente video: [video con dos nodos](#)